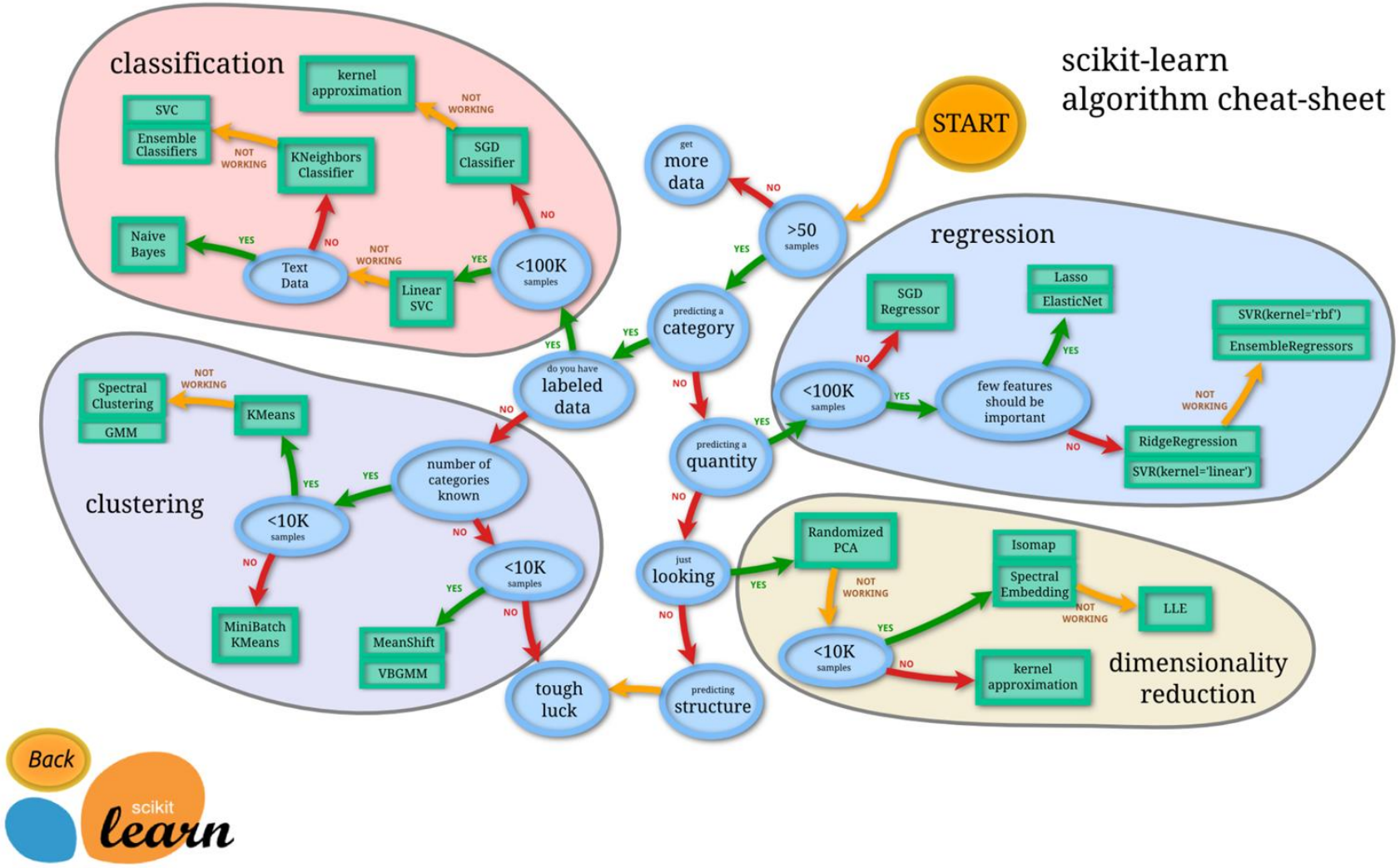
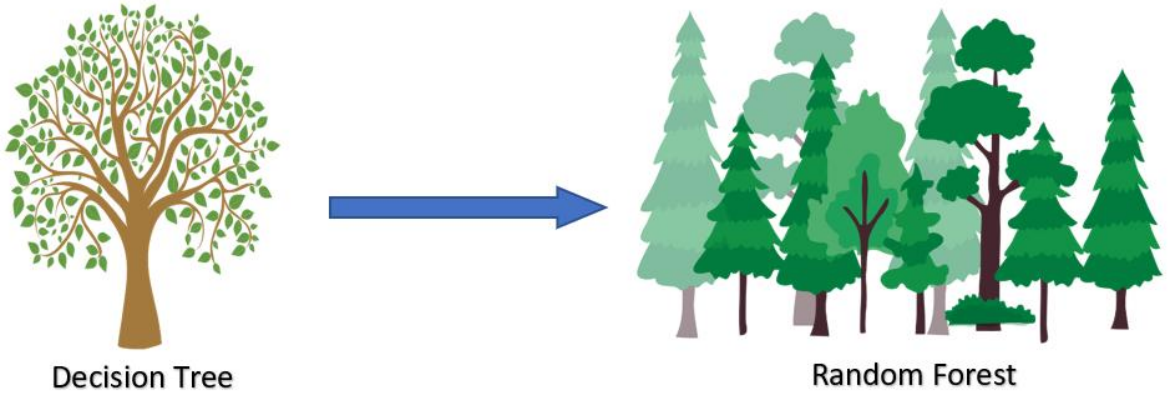
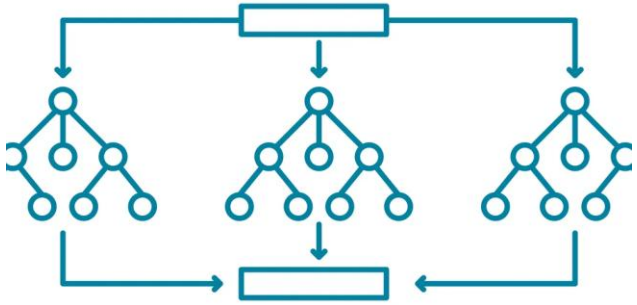
Machine learning - Guideline



Random Forest - Introducción

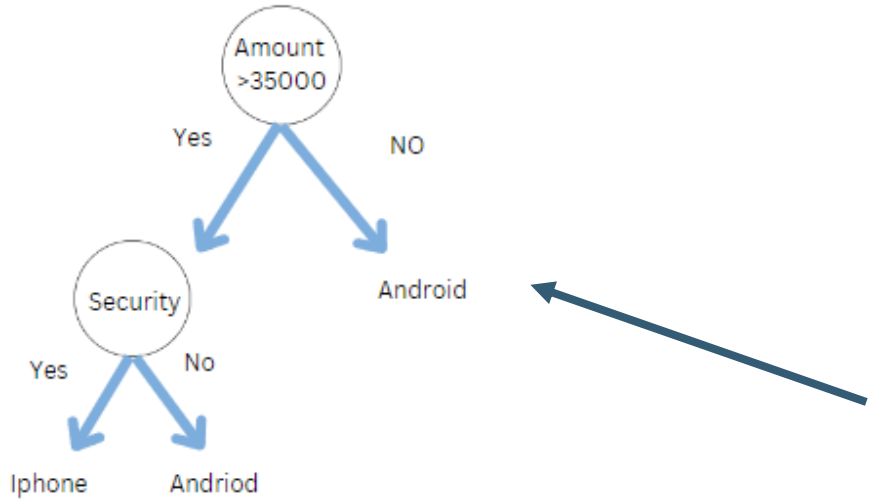
- ¿Por qué necesitamos un **bosque aleatorio**?
- R/ Similar al árbol de decisión. Random Forest también es un algoritmo de aprendizaje automático supervisado. Se utiliza ampliamente en **clasificación** y **regresión**.
- El árbol de decisión tiene un problema de sobreajuste.





El algoritmo Random Forest aborda este problema mediante la creación de múltiples árboles de decisión y la combinación de sus predicciones para generar predicciones más precisas.

Random Forest - Técnicas de Ensamble







En un árbol de decisión, solo tenemos **un árbol** para responder la pregunta.

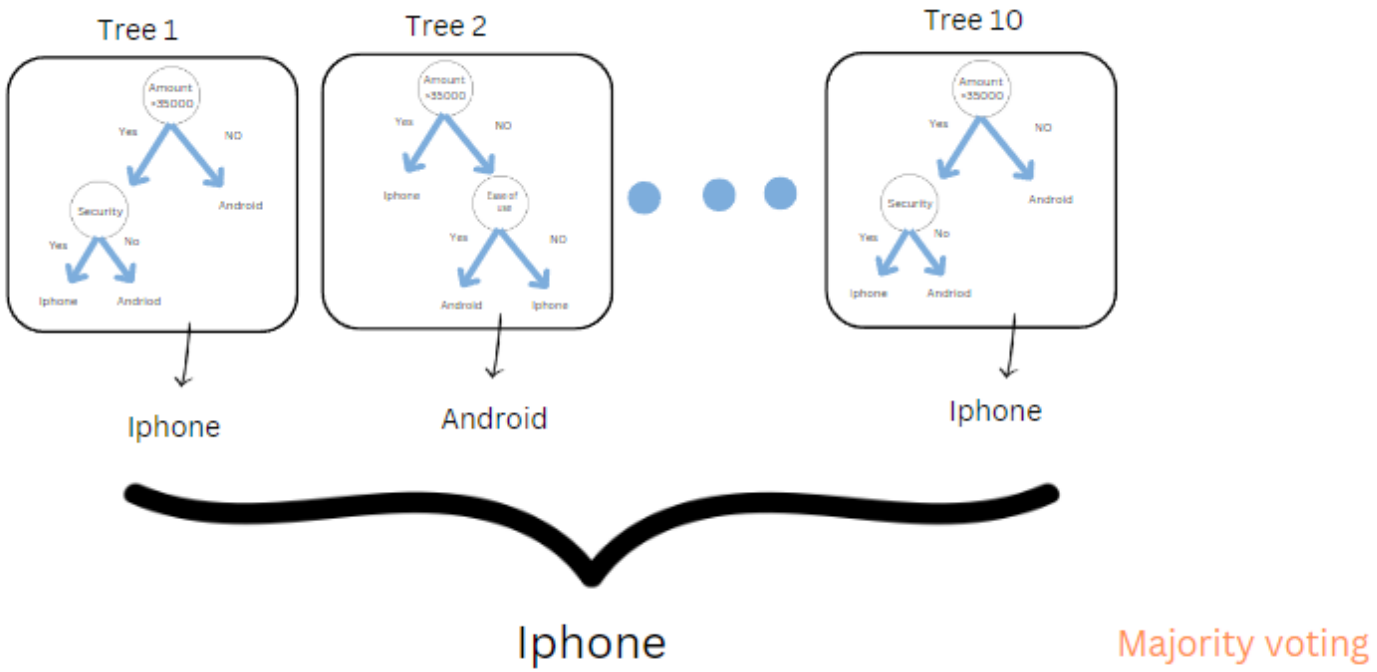
Ejemplo: Digamos que queremos comprar un teléfono.

Este árbol de decisiones se puede utilizar para decidir si debemos comprar un iPhone o un teléfono Android.

Sin embargo, en la vida real, **¿hacemos lo mismo, solo le pedimos a una persona recomendaciones para comprar un teléfono? Definitivamente no.**

Usualmente le preguntamos a varias personas como familiares, amigos, expertos y vendedores.

Random Forest - Técnicas de Ensamble

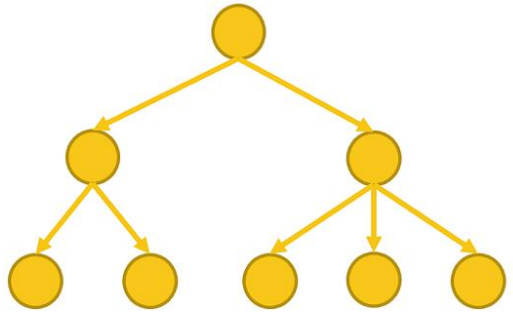


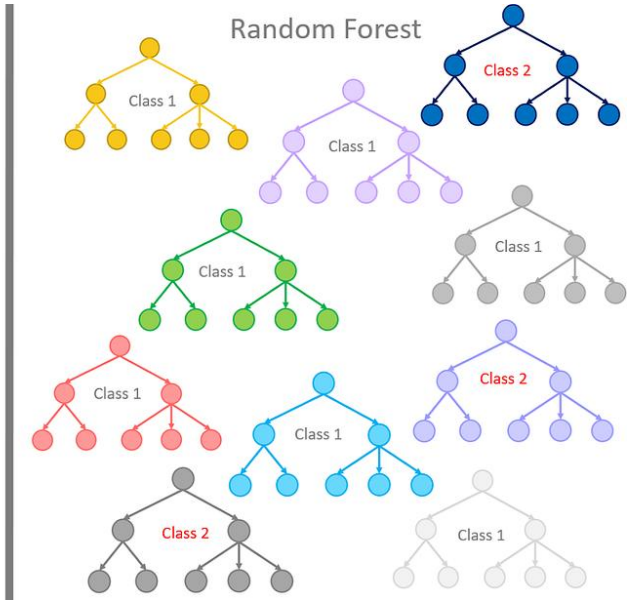
Suponga que cada árbol es como cada persona. Entonces, si le pregunta a **10** personas (**8** personas dijeron iPhone y **2** personas dijeron Android). En la **clasificación de bosque aleatorio**, el resultado final se basa en la **votación por mayoría**. **Compraremos un iPhone**.

Nota: ¿y si estamos construyendo un modelo de regresión?

Random Forest - Técnicas de Ensamble

- Llamamos al proceso de **combinación de varios modelos de árbol de decisión** un **bosque aleatorio**.
- Pero, la pregunta es, ¿podemos combinar múltiples modelos, como logístico, naïve Bayes y KNN, y si es así, cómo lo llamamos? Lo llamamos **Bagging** y **Boosting**.
- Son dos métodos de conjunto utilizados en el aprendizaje automático para **mejorar el rendimiento de un solo modelo al combinar las predicciones de múltiples modelos**.





Random Forest - Técnicas de Ensamble

BAGGING:

Combina cualquier modelo, como knn, naive Bayes, logístico, etc.

Sin embargo, el resultado será el mismo porque la entrada de datos será la misma para todos los modelos.

Para manejar eso, usaremos un agregador **Tipo Bootstrap**. Por ejemplo, si tenemos 10 modelos, cada modelo entrena en un subconjunto diferente de los datos de entrenamiento.

La **predicción final** suele ser el promedio o el voto mayoritario de las predicciones de todos los modelos.

BOOSTING:

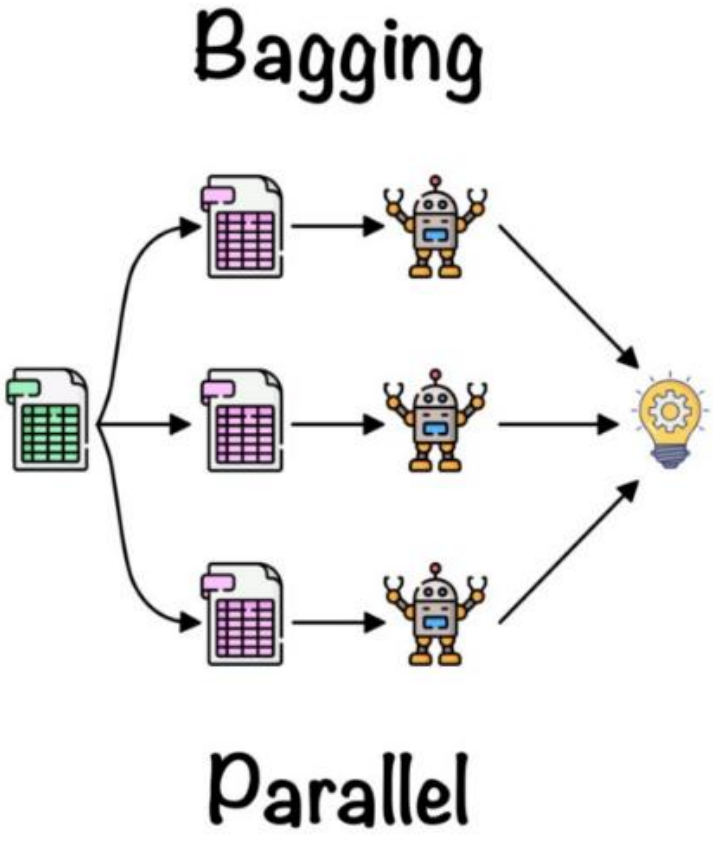
Produce aprendices (Learners) fuertes al combinar a los débiles siguiendo un **entrenamiento secuencial**, i.e., cada modelo trata de corregir los errores del modelo anterior.

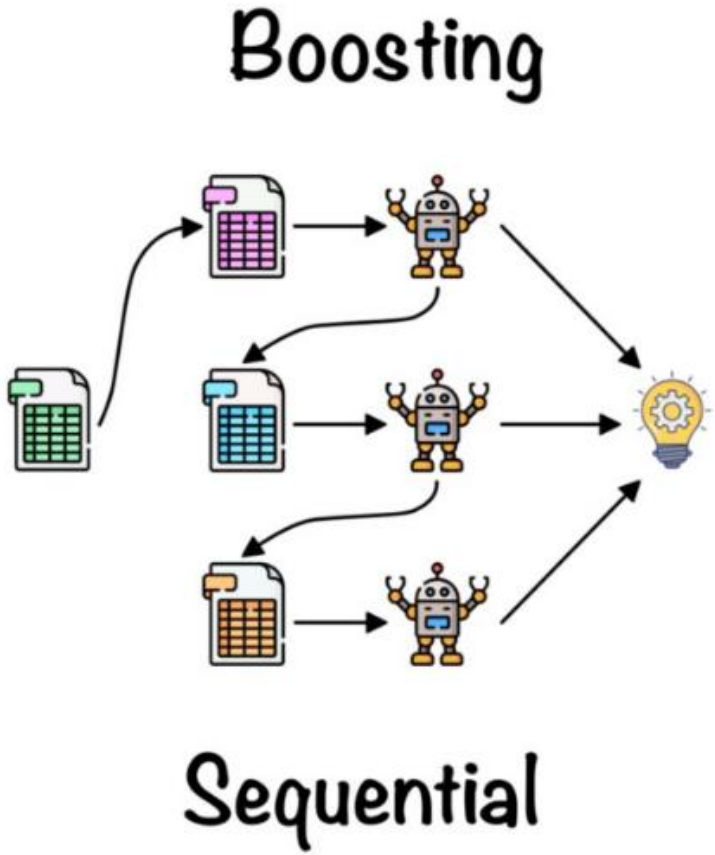
Los modelos sucesivos dependen del modelo anterior.

Tipos de algoritmos de boosting:

1. **Adaboosting**
2. **Gradient boosting**
3. **XGboost**

Random Forest - Técnicas de Ensamble

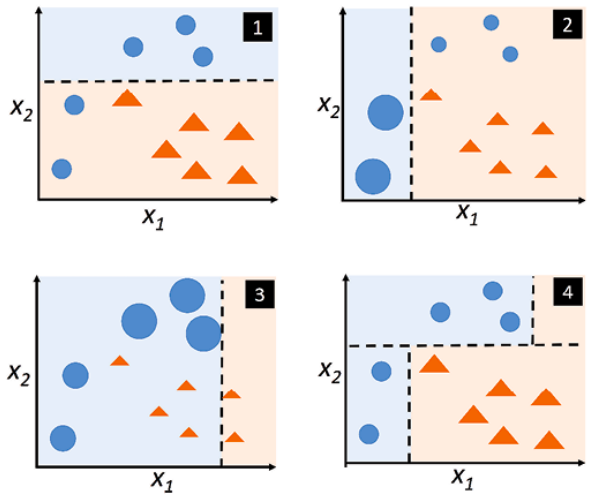




¿Con qué principio funcionan los random forest?

Random Forest - Técnicas de Ensamble

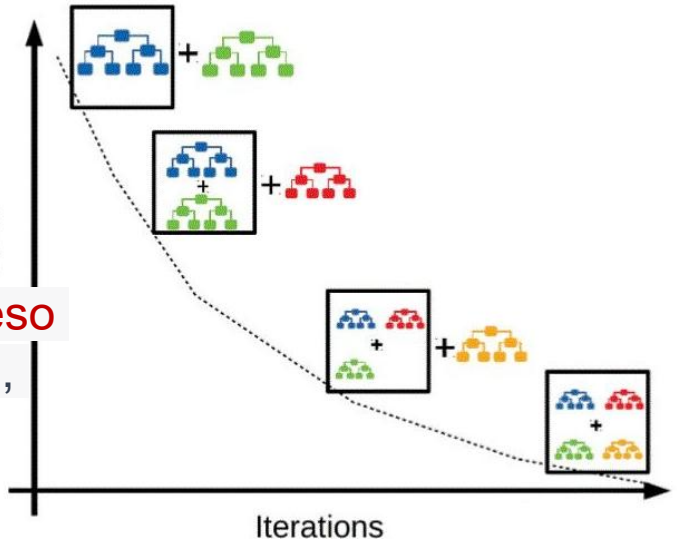
ADABOOST:



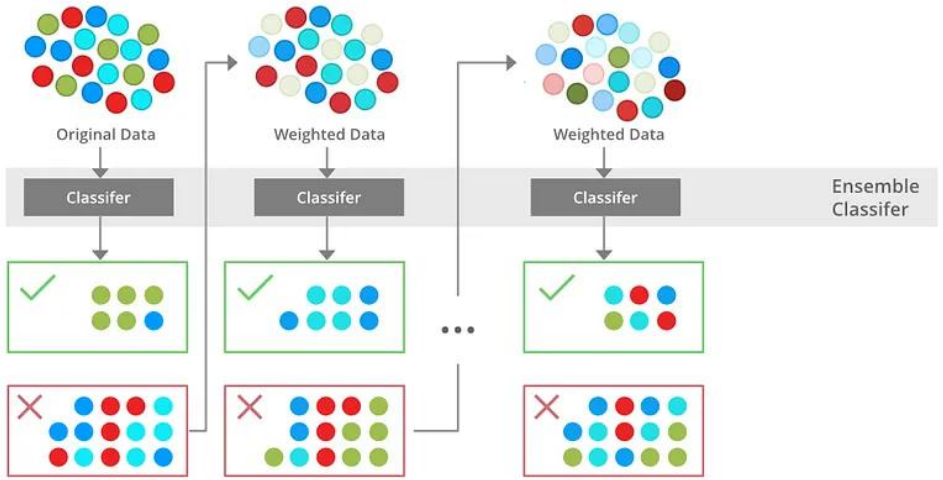
En cada iteración, se asigna un peso a cada instancia de entrenamiento, lo que permite que las instancias mal clasificadas tengan un mayor peso.

GRADIENT BOOST:

Utiliza el gradiente descendente para minimizar una función de pérdida. En cada iteración, se ajusta un modelo débil a los residuos del modelo anterior.



XG BOOST:



- Regularización
- Muestreo de columnas
- Manejo de datos faltantes

Random Forest - Técnicas de Ensamble





Ejemplo:

- Suponga que desea **comprar una casa**, ¿simplemente entrará en una inmobiliaria y comprará la primera casa que vea, o según el consejo de su corredor, comprará una casa recomendada? Es muy poco probable.

Acción:

- Probablemente navegará por algunos **portales web**, verificando el área, la cantidad de habitaciones, las instalaciones, el precio, etc. También probablemente le pedirá su opinión a sus amigos y colegas. En resumen, no llegarás a una conclusión directamente, sino que tomarás una decisión considerando también las opiniones de otras personas.

Técnica a implementar:

¿Bagging o Boosting?

Random Forest - ¿Cómo funciona?

Paso 1: ¿Cómo se usa un conjunto de datos de entrenamiento completo para construir varios árboles?

R/ Bootstrap con reemplazo

¿Qué es Bootstrap? - Creación de varios subconjuntos a partir del conjunto de datos de entrenamiento real.

Filas:

Cuando decimos con el reemplazo, en un subconjunto, podemos tener la misma fila varias veces.

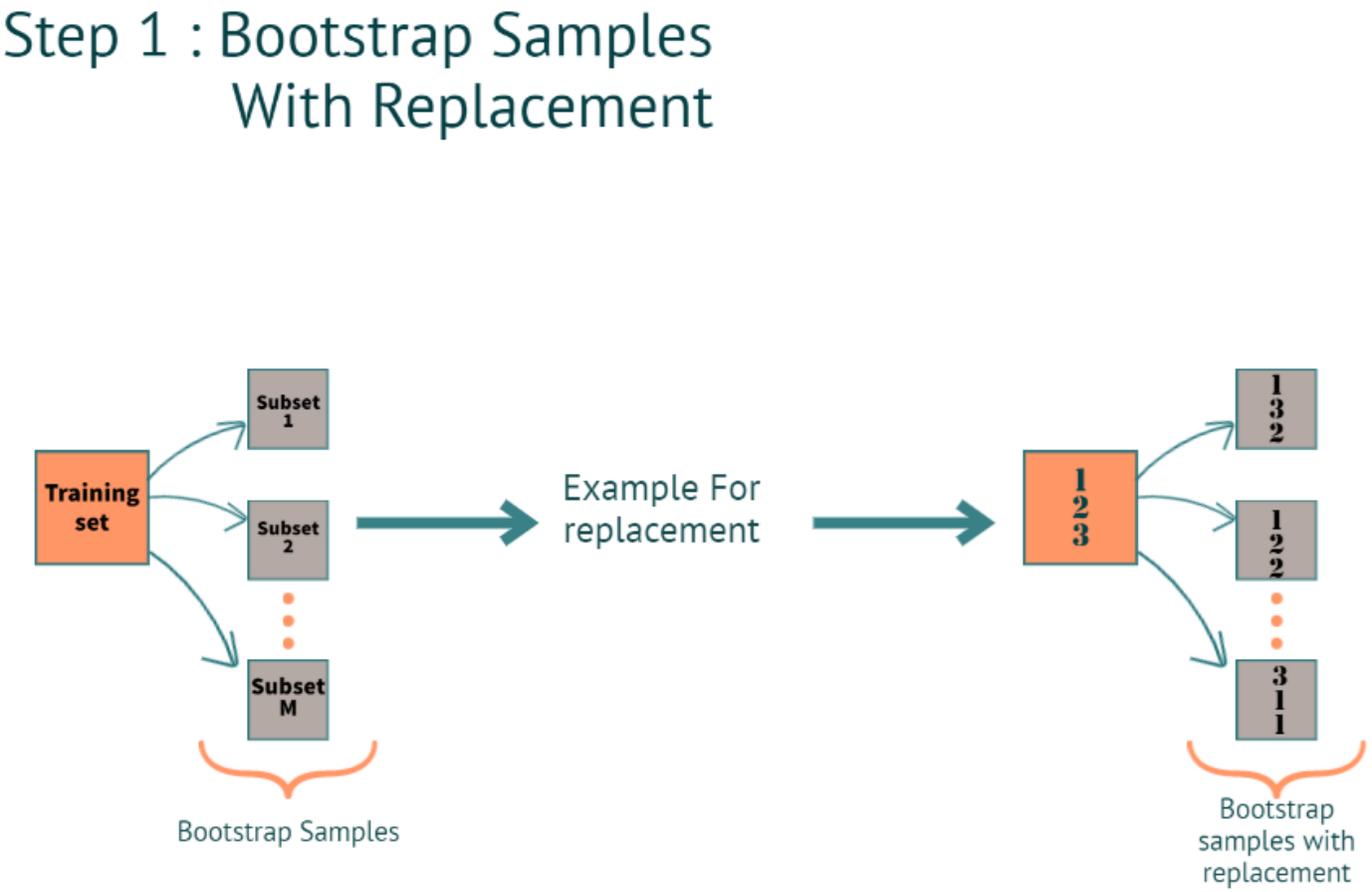
Columnas:

1. Para la **clasificación**, es una raíz cuadrada del número total de características.

2. **Regresión**: número total de características y dividirlas por 3.

Random Forest - ¿Cómo funciona?

Paso 1: ¿Cómo se usa un conjunto de datos de entrenamiento completo para construir varios árboles?



Random Forest - ¿Cómo funciona?

Paso 2: Se pueden construir múltiples árboles de decisión siguiendo los pasos

Construimos un árbol de decisión para cada subconjunto.

¿Cómo pudimos construir los árboles de decisión desde cero?

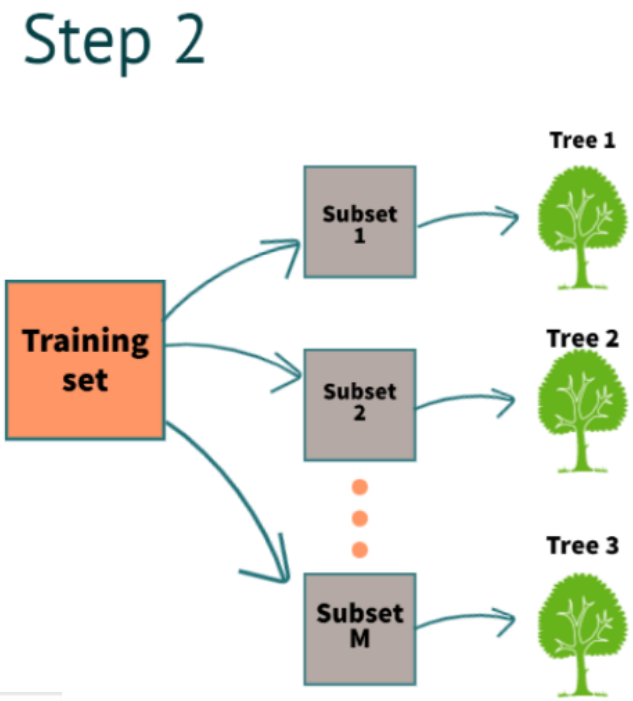
Para construir un árbol de decisión, tenemos que usar dos métodos.

1. **Gini**

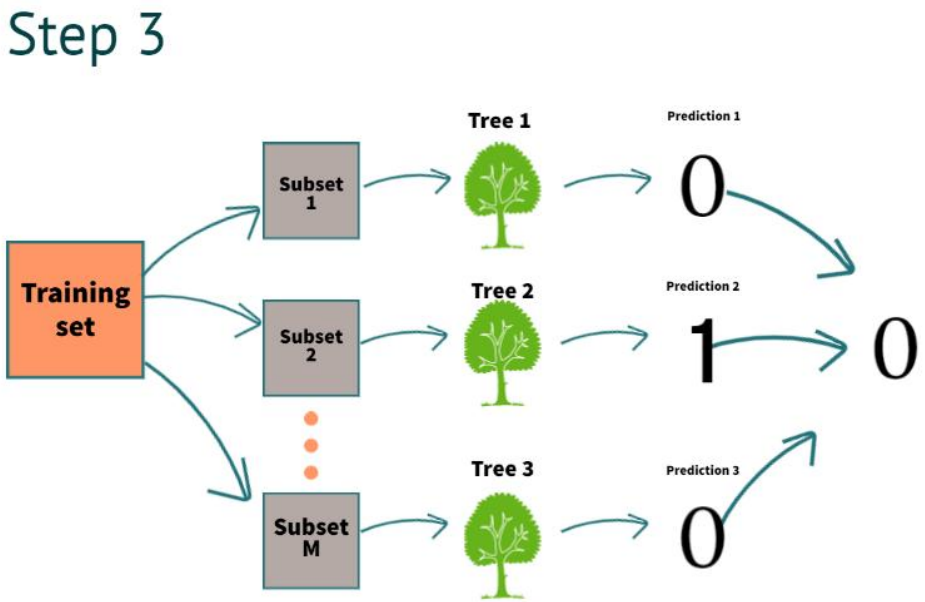
$$I_G = 1 - \sum_{j=1}^{c} p_j^2$$

2. Entropía y ganancia de información

$$I_H = -\sum_{j=1}^{c} p_j log_2(p_j)$$



Random Forest - ¿Cómo funciona?



Paso 3: Uso de modelos de árboles múltiples para predecir el resultado, ¿cuál es el proceso de predicción del resultado?

Supongamos que tenemos nueva información para la predicción

Salario —> 10.000 USD
Propiedad —> No
Aprobación de préstamo —> ?

El modelo lo predice como **“0”**. Al combinar todas las predicciones del árbol de decisiones anteriores, como puede ver en la imagen.

¿Qué queremos decir con combinar la predicción de todos los árboles?

Para entenderlo, vamos al paso cuatro.

Random Forest - ¿Cómo funciona?

Paso 4: cuando un modelo finaliza un resultado de regresión o clasificación, ¿cómo se llama?

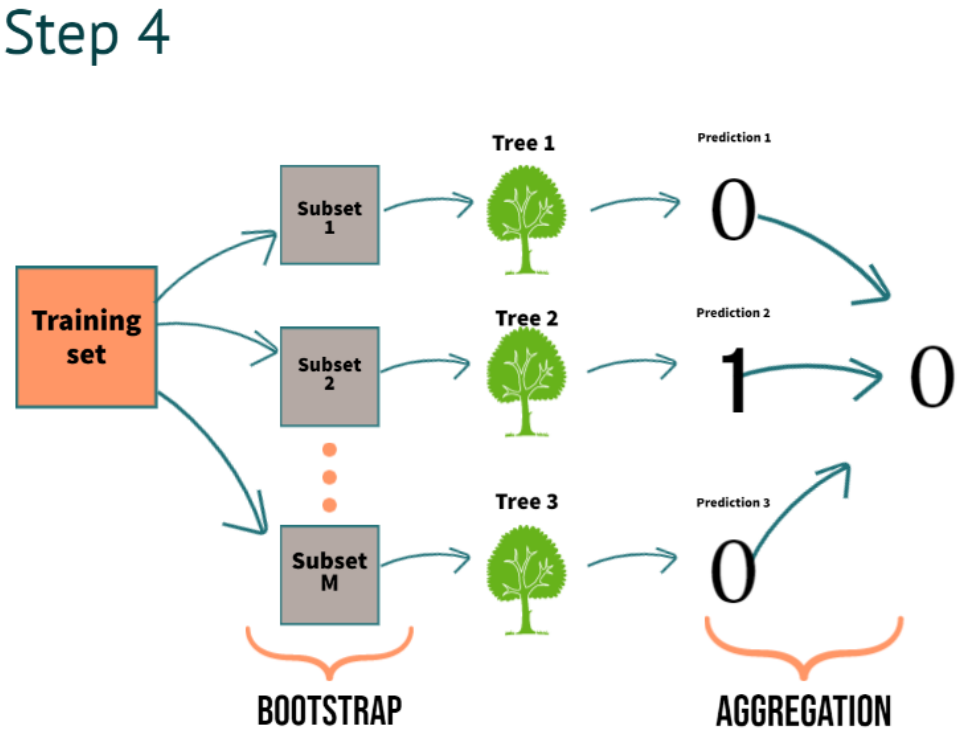
Para la clasificación, utilizamos la **votación por mayoría.**

Para la regresión, usamos el **promedio**

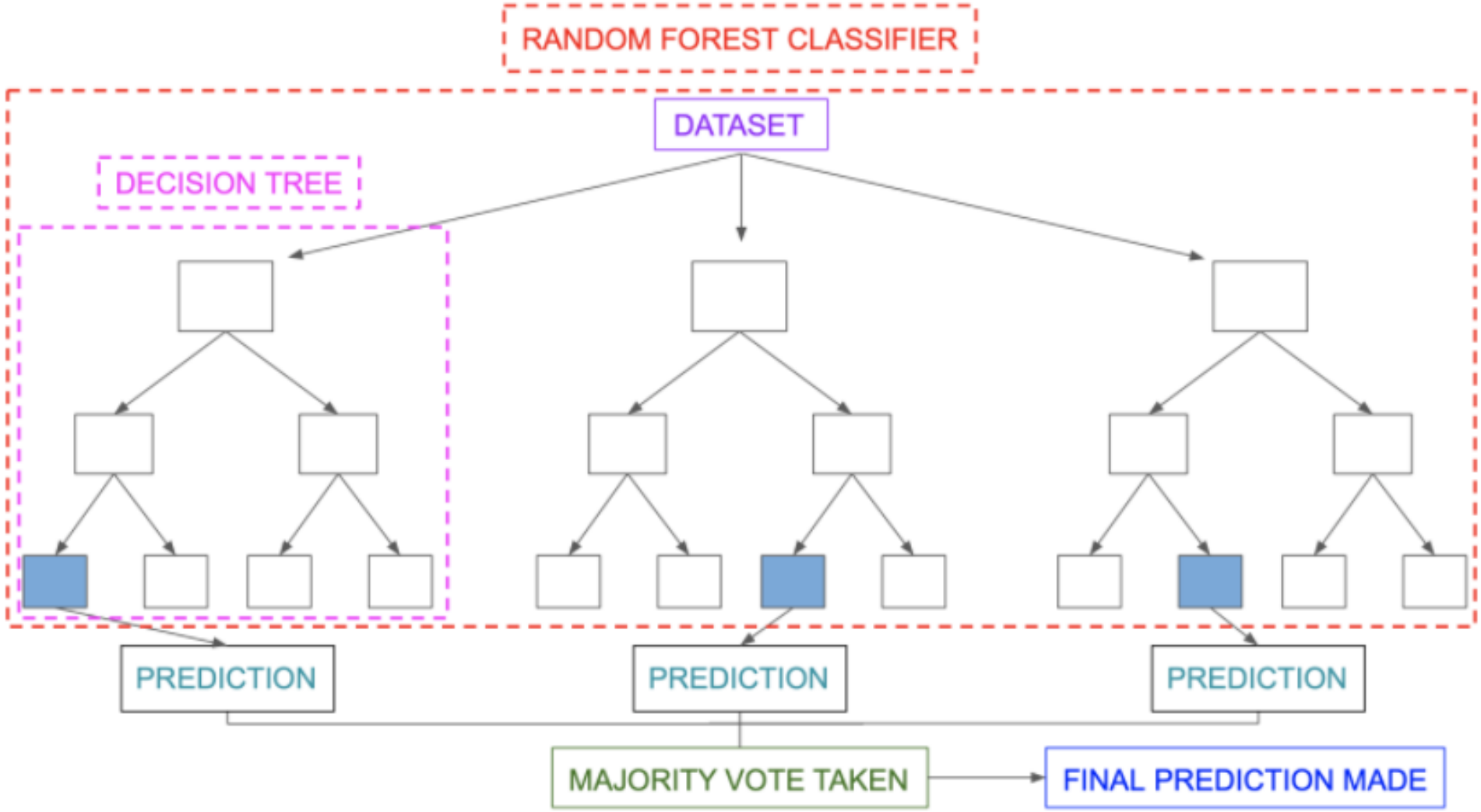
Con esto, entendemos de qué se trata exactamente un **Bootstrap**.

¿Cómo se beneficia el RF?

Reduce la varianza. Esto ayuda a construir modelos robustos, que funcionan bien incluso con datos ocultos.



Random Forest - ¿Cómo funciona?



Diferencias entre DT y RF

- Normalmente sufren el problema del sobreajuste si se les permite crecer hasta su máxima profundidad.
- Es más rápido en el cálculo.
- Cuando un conjunto de datos con características se toma como entrada de un árbol de decisión, formulará un conjunto de reglas para hacer predicciones.



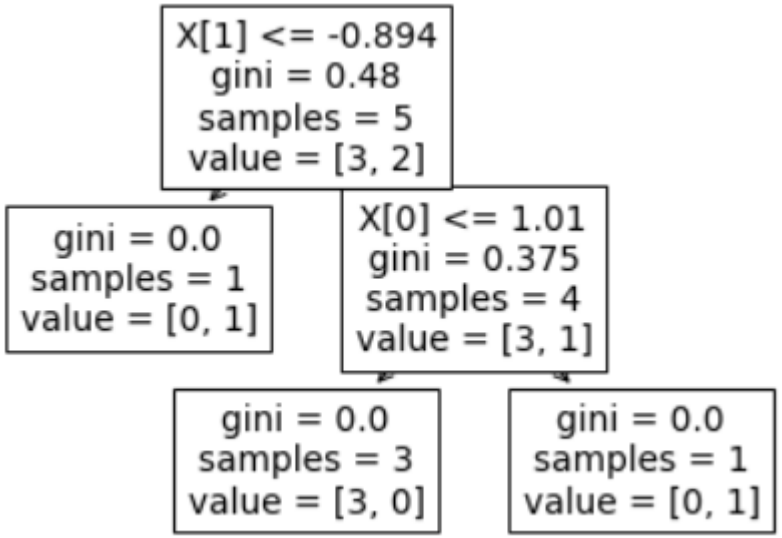
- Utilizan el método de **Bagging**. Crea un subconjunto del conjunto de datos original, y el resultado final se basa en la clasificación mayoritaria y, por lo tanto, se soluciona el problema del sobreajuste.
- Es comparativamente más lento.
- Random forest selecciona aleatoriamente las observaciones, construye un árbol de decisión y se toma el resultado promedio. No utiliza ningún conjunto de fórmulas.

Los bosques aleatorios son mucho más exitosos que los árboles de decisión sólo si los árboles son diversos y aceptables.

Random Forest - Importancia de la característica

Otra gran cualidad de este impresionante algoritmo es que también se puede utilizar para la **selección de características**. Podemos usarlo para **saber la importancia de la característica**.

Ejemplo: Tenemos el siguiente árbol de decisión.



Aquí tenemos **dos columnas [0 y 1]**, para calcular la importancia de la característica de [0] necesitamos **encontrar** aquellos nodos donde ocurrió la división debido a esta columna [0].

En este conjunto de datos, solo tenemos 1 nodo para la columna [0] y la columna [1].

Random Forest - Importancia de la característica

Objetivo:
De todos los nodos, encontraremos la importancia de la característica y luego la dividiremos por la importancia de la característica de todos los nodos.
Solución:
Para calcular la importancia de un nodo utilizaremos esta fórmula:

$$n_i = \frac{N_t}{N}[impurity\ -\ (\frac{N_{t(right)}}{N_t} * right\ impurity) - (\frac{N_{t(left)}}{N_t} * left\ impurity)]$$

$where\ N_t\ is\ number\ of\ rows\ that\ particular\ node\ has$

$N\ is\ the\ total\ number\ of\ rows\ present\ in\ data$

$Impurity\ is\ our\ gini\ index\ value$

$N_{t(right)}\ is\ number\ of\ nodes\ in\ right\ node$

$N_{t(left)}\ is\ number\ of\ nodes\ in\ left\ node$

Random Forest - Importancia de la característica

Calculemos la importancia de la característica del **primer nodo** en nuestro árbol de decisión.

$$
\begin{aligned}
&= \frac{5}{5}[0.48 - (\frac{4}{5} * 0.375) - (\frac{1}{5} * 0)] \\
&= [0.48 - (0.8 * 0.375)] \\
&= 0.48 - 0.30 \\
&= 0.18
\end{aligned}
$$

Del mismo modo, calcularemos esto para el **segundo nodo**:

$$
\begin{aligned}
&= \frac{4}{5}[0.375 - (\frac{1}{5} * 0) - (\frac{1}{5} * 0)] \\
&= 0.8 * 0.375 \\
&= 0.30
\end{aligned}
$$

Random Forest - Importancia de la característica

Ahora calculemos la **importancia de las características [0] y [1]**. Esto se puede calcular como:

$$fi_i = \frac{\sum_{j:node\ j\ splits\ on\ feature\ i} ni_j}{\sum_{k \in all\ nodes} ni_k}$$

$$\text{Feature } 0 = \frac{0.3}{0.3 + 0.18} = 0.625$$

$$\text{Feature } 1 = \frac{0.18}{0.18 + 0.3} = 0.375$$

Para calcular la importancia de las características utilizando Random Forest, simplemente **tomamos un promedio de todas las importancias de las características de cada árbol**.

Random Forest - Pros y Contras

Ventajas:

- Es flexible.
- Se puede usar para clasificación y regresión.
- Puede considerarse un algoritmo útil porque produce mejores resultados incluso sin ajuste de hiperparámetros.
- Los parámetros son bastante sencillos, son fáciles de entender y tampoco hay muchos de ellos.
- El bosque aleatorio ayuda a superar el overfitting al combinar muchos árboles de decisión que eventualmente nos darán un sesgo bajo y una varianza baja.

Desventajas:

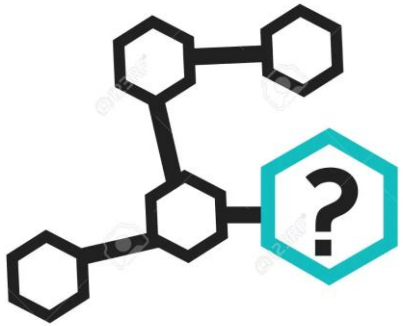
- Debido a la gran cantidad de árboles, el algoritmo tarda mucho en entrenarse, lo que lo hace lento e ineficaz para las predicciones en tiempo real.
- bastante lentos para crear predicciones una vez que están entrenados.

Random Forest - Datos Faltantes

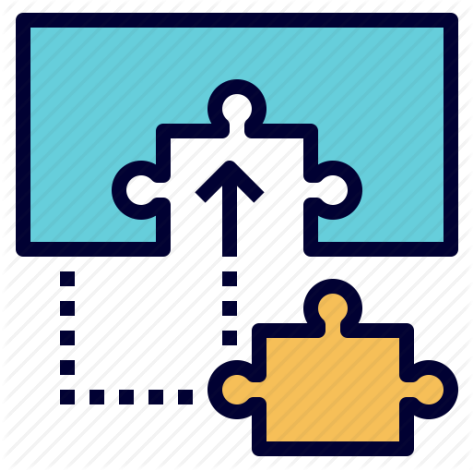
Random Forest para la **imputación de datos** es una forma emocionante y eficiente de imputación, y tiene casi todas las cualidades de ser la **mejor técnica de imputación**.

Analizaremos dos de estos métodos:

- Miss Forest.
- Mice Forest.







Random Forest - Datos Faltantes

MISS FOREST

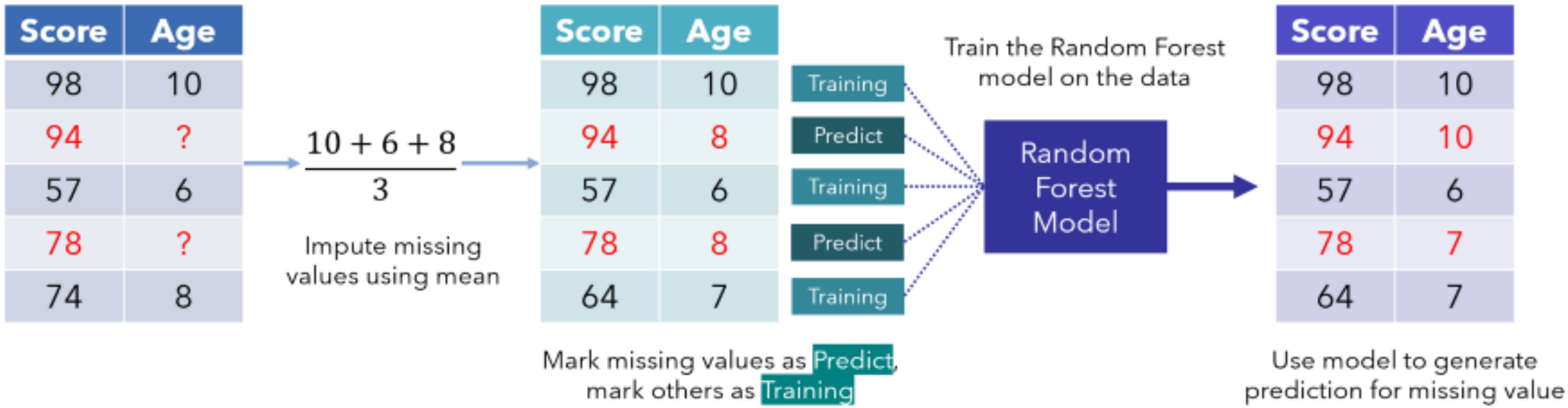
Si necesita **precisión**, entonces esto es lo que debe usar. Una técnica de imputación iterativa impulsada por Random Forest para imputar datos con precisión. **¿Entonces, cómo funciona?**

Paso 1: Primero, los valores faltantes se completan con la media de las columnas respectivas para datos continuos y más frecuentes para datos categóricos.

Paso 2: El conjunto de datos se divide en **dos partes**: datos de entrenamiento que consisten en las variables observadas y la otra son los datos faltantes que se utilizan para la predicción. Estos conjuntos de entrenamiento y predicción luego se alimentan a Random Forest y, posteriormente, **los datos pronosticados se imputan en los lugares apropiados**. Después de imputar todos los valores, se completa una iteración.

Paso 3: El paso anterior se repite hasta que se alcanza una condición de parada. El proceso de iteración garantiza que el algoritmo funcione con datos de mejor calidad en iteraciones posteriores. El proceso continúa hasta que aumenta la suma de las diferencias al cuadrado entre la imputación actual y la anterior o se alcanza un límite de iteración específico. **Por lo general, se necesitan de 5 a 6 iteraciones para atribuir bien los datos.**

Random Forest - Datos Faltantes



Random Forest - Datos Faltantes

Ventajas de Missing Forest

- Puede trabajar con datos mixtos, tanto numéricos como categóricos.
- Miss Forest puede manejar valores atípicos, por lo que no es necesario escalar características.
- Los bosques aleatorios tienen una selección de características inherente, lo que los hace resistentes a los datos ruidosos.
- Puede manejar la **no linealidad** en los datos.

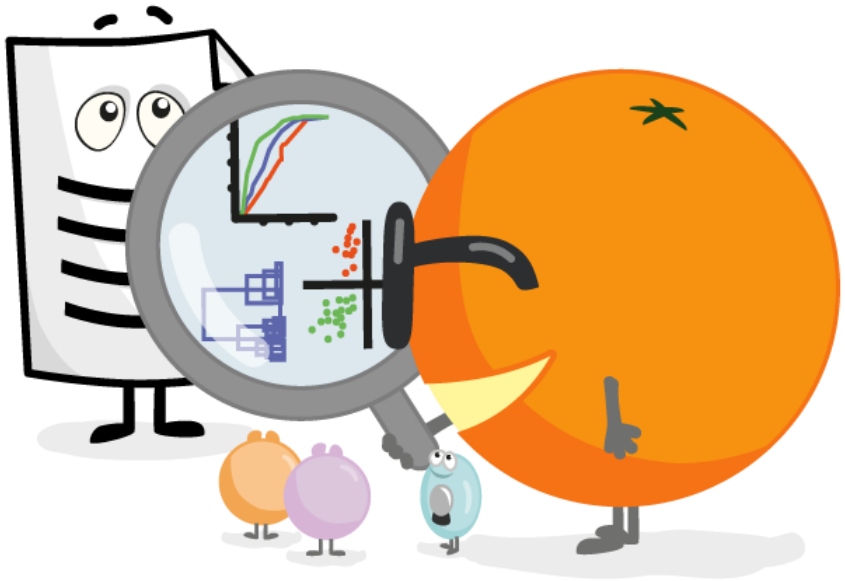
Desventajas de Missing Forest

- Es necesario construir varios árboles para cada iteración, y se vuelve costoso desde el punto de vista computacional cuando aumenta la cantidad de predictores y observaciones.
- Además, es un algoritmo, no un objeto modelo, lo que significa que debe ejecutarse cada vez que se imputan datos, lo que podría ser problemático en algunos entornos de producción.

Random Forest - Hiperparámetros

Hiperparámetros para aumentar el poder predictivo

- *n_estimators:* número de árboles que construye el algoritmo antes de promediar las predicciones.
- *max_features:* número máximo de características que el bosque aleatorio considera dividir en un nodo.
- *mini_sample_leaf:* determina el número mínimo de hojas necesarias para dividir un nodo interno.
- *criterio:* ¿Cómo dividir el nodo en cada árbol? (Entropía/impureza de Gini/pérdida de registro)
- *max_leaf_nodes:* máximo de nodos hoja en cada árbol.



RAPIDMINER